

Questão 01:

Qual é a saída?

R: false e 9007199254740992

O problema é detectado na compilação, na execução ou nunca?

R: O problema nunca é detectado.

```
hugo@hortela-linux: ~/Área de trabalho/Faculdade/6S/paradigmas/2b/exec01
hugo@hortela-linux:~/Área de trabalho/Faculdade/6S/paradigmas/2b/exec01$ node q1.js
false
9007199254740992
```

Questão 02:

Qual é a saída:

R: 4 e 6.

O problema é detectado na compilação, na execução ou nunca?

R: Não existe erro, apenas a forma de representar os caracteres são diferentes.

```
hugo@hortela-linux:~/Area de trabalho/Faculdade/6S/paradigmas/2b/exec01$ python3 q2.py
4 6
```

Questão 03:

Qual é a saída?

R: 0

O problema é detectado na compilação, na execução ou nunca?

R: O problema nunca é encontrado.

```
main.go x
1 package main
2
3 import "fmt"
4
5 func main() {
6     var b byte = 255
7     b++
8     fmt.Println(b)
9 }
10
```

Console

0

Questão 04:

Qual é a saída?

R: 0 e Exception.

O problema é detectado na compilação, na execução ou nunca?

R: Sim, o problema é identificado na execução.

```
Main.java x
1 public class Main {
2     public static void main(String[] args) {
3         int[] v = new int[3];
4         System.out.println(v[0]);
5         System.out.println(v[3]);
6     }
7 }
```

Console I/O AI Agent Exit 1

Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3
at Main.main(Main.java:5)

0

Debug this error

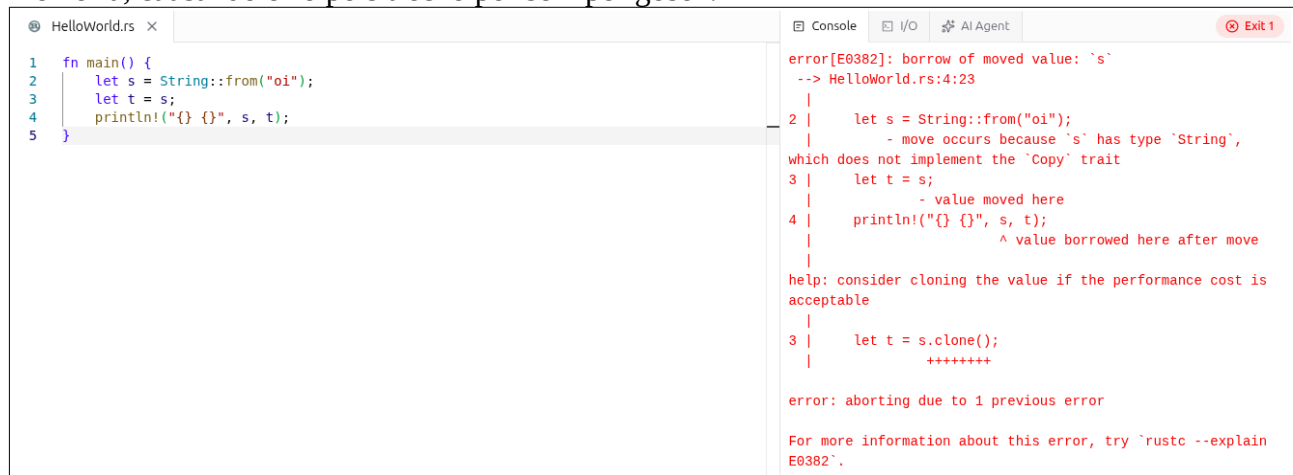
Questão 05:

Qual é a saída?

R: (nenhuma saída pois não compila)

O problema é detectado na compilação, na execução ou nunca?

R: O problema é encontrado na compilação. O Rust tem uma filosofia rigorosa na segurança de memória, causando erro pois trecho por ser “perigoso”.



```
1 fn main() {
2     let s = String::from("oi");
3     let t = s;
4     println!("{}", s, t);
5 }
```

error[E0382]: borrow of moved value: `s`
--> HelloWorld.rs:4:23

```
2 |     let s = String::from("oi");
  |         - move occurs because `s` has type `String`,
  |         which does not implement the `Copy` trait
3 |     let t = s;
  |         - value moved here
4 |     println!("{}", s, t);
  |                  ^ value borrowed here after move
```

help: consider cloning the value if the performance cost is acceptable

```
3 |     let t = s.clone();
  |               +++++++
```

error: aborting due to 1 previous error

For more information about this error, try `rustc --explain E0382`.

Questão 06:

Qual é a saída?

R: 1065353216

O problema é detectado na compilação, na execução ou nunca?

R: Nunca é detectado.

```
hugo@hortela-linux:~/Área de trabalho/Faculdade/6S/paradigmas/2b/exec01$ gcc q6.c -o q6 && ./q6
1065353216
```